

Exercise 1: Student Grade Manager

Create a program that manages student grades using One Of Collections

1. Create a Collection with these grades: 85, 92, 78, 95, 88, 70, 100, 65
2. Print the collection, Count, first and last grade
3. **Sort** the grades ascending, then print
4. Get the first grade above 90
5. Get all grades below 75 (failing grades)
6. Remove all failing grades (below 75)
7. Check if any grade equals 100
8. Create a List<string> where each grade becomes "Grade: X"

Exercise 2: Leaderboard

Create a leaderboard that automatically sorts players by score.

1. Add: 500="Ahmed", 200="Sara", 800="Ali", 350="Mona"
2. Print all entries (they should be sorted by score automatically)
3. Access the **first key** and **first value**
4. Check if score 500 exists
5. Safely get the player with score 999
6. **Remove** the player with score 200 and print the updated list

Exercise 3: Phone Book

Build a phone book application.

1. Create a **Collection** with 4 contacts (name → phone number)
2. Add a new contact using **[] syntax** (add or update)
3. Try adding a duplicate using **.Add()** — catch the exception and print the error
4. Try adding a duplicate using **.TryAdd()** — print whether it succeeded
5. Search for a contact that doesn't exist
6. Get a contact with a fallback of "Not Found"
7. Print all **Keys** on one line, then all **Values** on another line

Exercise 4: Unique Email Validator

Use Collection to manage unique email addresses.

1. Create a **HashSet<string>** with a **case-insensitive comparer**: `new HashSet<string>(StringComparer.OrdinalIgnoreCase)`
2. Add these emails: "ahmed@test.com", "AHMED@test.com", "sara@test.com", "Sara@Test.Com"
3. Print Count — how many are actually stored? Explain why.
4. Create two sets: **Set A** = {1,2,3,4,5} and **Set B** = {4,5,6,7,8}
5. Print the result of: **UnionWith**, **IntersectWith**, **ExceptWith**
6. Use **IsSubsetOf** to check if {1,2} is a subset of Set A

Exercise 5: Print Queue Simulator

Simulate a printer queue

Create a **Queue<string>** and enqueue 5 documents: "Report.pdf", "Invoice.pdf", "Letter.docx", "Resume.pdf", "Photo.jpg"

1. Print the queue contents and Count
2. Use **Peek** to see which document will print next (without removing)
3. Process the queue: **Dequeue** each document and print "Printing: [name]"
4. Try **TryDequeue** on the now-empty queue — what happens?

Exercise 6: Browser History (Undo)

Simulate browser back/forward

Create a **Stack<string>** for browser history

1. Push 5 URLs: "google.com", "github.com", "stackoverflow.com", "youtube.com", "claude.ai"
2. Use **Peek** to see the current page (top of stack)
3. Press "back" 3 times using **Pop** — print each page you leave
4. Print the current page after going back
5. Try **TryPop** on an empty stack — what happens?